Auditable Compositional Question Answering over UK Public Procurement

Anonymous *anonymous@example.com*
University College London

Abstract

The United Kingdom publishes hundreds of thousands of contract award notices, yet the public cannot reliably analyse them in natural language, because the questions that matter are exhaustive aggregations where a fluent wrong answer is worse than none. We introduce, to the best of our knowledge, the first systematic benchmark and executable framework for auditable compositional question answering over UK public procurement. The task ships complete. A convention-explicit knowledge graph of 215,221 contract awards, a benchmark of 12,828 questions each carrying an executable gold program, a typed compositional operator algebra that formalises real procurement analytics, refusal on ambiguous, unsupported and empty questions as scored behaviour, and answer oracles validated by an independent second implementation at 99.88 percent over 14,770 audited answers. Because no annotator pool can label compositional procurement programs at scale, training uses a verified bootstrap. Models propose candidate programs and deterministic checks, compilation, grounding, execution and oracle agreement, decide what may be learned from. An 8-billion-parameter local model trained this way reaches 85.65 percent against 69.76 for the cloud teacher that generated its training data (paired McNemar $p < 10^{-15}$), and 78.31 percent on a sealed challenge set with an independent surface grammar, a constraint-necessity audit, and a 28-point margin over the same teacher. Transplanted to WikiTableQuestions by exchanging only the data-representation layer, the recipe lifts a 22.5 percent zero-shot floor to 44.4 with answer-only supervision and 51.8 with gold programs on the official sealed test, while a four-way attribution audit shows a substantial share of the apparent expressiveness gap to be representation and compilation debt rather than missing reasoning capability. We claim no state of the art. The claim is a task built end to end, and audits strong enough to say precisely what its system can and cannot yet do.

1 Introduction

Public bodies in the United Kingdom publish hundreds of thousands of contract award notices every year. Journalists and auditors ask analytical questions of this record. How much did a council spend on cleaning in 2023. Which supplier received the most awards under a procurement category. Did spending rise after a policy change. An answer to such a question carries the authority of official data, so a fluent but wrong answer is worse than no answer. A system for this task must produce answers that a reader can trace to specific records, and it must refuse when the data cannot support one.

Large language models read these questions well and answer them unverifiably. Nothing in generated text separates a genuine aggregation over all matching records from a plausible guess over a retrieved sample. Retrieval augmentation (Lewis et al., 2020) does not close this gap, because the questions are dominated by exhaustive computation. A count over every contract of every London borough does not fit in a context window. On our development set a retrieval baseline answers 31.5 percent of questions while an untrained 8B model that is forced to emit an executable plan answers 61.5 percent. Forcing an executable plan is worth more than retrieval before any training takes place.

The standard route to a competent small model is distillation from a large one. That route assumes the teacher can be trusted, and on this task it cannot. The strongest cloud pipeline we evaluate answers 69.76 percent

of held-out questions. Imitating it transfers its errors together with its competence. Filtered self-training methods (Zelikman et al., 2022; Gulcehre et al., 2023) replace trust with a pass criterion, but the criteria in use are proxies. A SQL query can execute cleanly and compute the wrong thing. A model can agree with itself and be wrong. Outputs that pass for the wrong reason enter the training pool silently, and none of these methods measures how often that happens.

We introduce, to the best of our knowledge, the first systematic benchmark and executable framework for auditable compositional question answering over UK public procurement data. Procurement knowledge graphs exist, and TheyBuyForYou (Soylu et al., 2022) built one across EU procurement for integration, anomaly detection and search. What has not existed is the task itself in usable form. That means a knowledge graph whose money and identity conventions are explicit enough to verify against, a benchmark whose every question carries an executable gold program, a typed program language that expresses real analytical demands, refusal as a scored behaviour, and an evaluation whose oracles are measured rather than assumed. We build all of it, and we train a system on it without large-scale manual annotation.

The training method follows from the setting. No annotator pool can label tens of thousands of compositional procurement programs, so supervision must come from models, and the task itself supplies a filter stronger than any proxy. Whether a plan compiles, grounds, executes, and matches an independently computed oracle are deterministic checks. We train only on outputs that pass all of them, whichever model produced them. An 8-billion-parameter local model trained this way reaches 85.65 percent on the held-out benchmark of 2,285 questions, against 69.76 percent for the cloud teacher that generated its training data. The gap is significant at $p < 10^{-15}$ under a paired McNemar test (McNemar, 1947) and survives a measured teacher noise floor of 1.1 points.

The recipe transfers. We rebuild only the data layer and move the system to WikiTableQuestions (Pasupat & Liang, 2015), a benchmark of crowd-written questions over web tables that we did not construct. The procurement checkpoint alone moves the zero-shot floor from 22.5 to 27.3 percent. The recipe moves it to 44.4 percent when supervision comes from final answers only, and to 51.8 percent when gold programs exist, both on the official sealed test under a single frozen-configuration run. The checkpoint accounts for 4.8 points of the transfer gain, while verified training accounts for the larger subsequent improvement.

We make five contributions, ordered from the task outward.

First, the task. We define auditable compositional question answering over UK public procurement, from raw releases to a convention-explicit knowledge graph of 215,221 contract awards, with refusal on ambiguous, unsupported and empty questions as first-class behaviour (Sections 3 and 4).

Second, the representation. A typed compositional operator algebra formalises real procurement analytics, set composition, guarded money aggregation, grouped comparison and multi-stage analysis, as closed but recursively compositional executable programs (Section 3).

Third, the evaluation foundation. A program-first benchmark of 12,828 questions with executable gold programs and plan-level split isolation, a sealed challenge set with an independent surface grammar and a constraint-necessity audit, and oracles validated by a second independent implementation at 99.88 percent over 14,770 audited answers (Sections 4 and 5.2).

Fourth, the training method the setting demands. A verified bootstrap in which deterministic execution checks, not teacher trust and not proxy rewards, decide what the model may learn from, requiring no manual program annotation (Section 5.1).

Fifth, the empirical case. The trained local system reaches 85.65 percent at home and 78.31 on the sealed challenge set, 28 points above its cloud teacher there, and the recipe transfers to WikiTableQuestions at the cost of a data layer, where an attribution audit separates representation debt from genuine language limits (Section 5.3).

We claim no state of the art. Recent specialised table systems score higher on WikiTableQuestions than we do. The claim is that a deterministic filter over an executable language turns weak local models into systems that beat their own teachers, across both the home benchmark and an independently constructed challenge set.

2 Related Work

The loop we use, generate candidates, filter them, retrain on the survivors, is standard. We do not claim it. The difference against each neighbour below lies on two axes. The first is what a candidate must prove before it becomes supervision. The second is what happens to candidates that pass wrongly. We state both for every cluster.

Procurement knowledge graphs. TheyBuyForYou (Soylu et al., 2022) integrated cross-national EU procurement into a knowledge graph and built services for data integration, anomaly detection and search. Later work organises procurement documents into navigable knowledge bases. These efforts are relevant because they establish that procurement data rewards graph treatment, and they are not enough for our purpose because none defines question answering as an executable task. They ship no gold programs, no compositional query language, no refusal semantics, and no independently-audited oracles, which are exactly the components a measurable QA task needs.

Filtered self-training. STaR (Zelikman et al., 2022) keeps sampled rationales whose final answer matches a gold label and retrains on them. Rejection-sampling fine-tuning (Yuan et al., 2023) and ReST (Gulcehre et al., 2023) generalise the pattern to reward thresholds and iterated grow-filter-train cycles. ExeSQL (Zhang et al., 2025) bootstraps text-to-SQL with execution success as the filter, and self-correction distillation (Zhu et al., 2026) transfers query generation and error-correction trajectories from a trusted teacher. All of these are relevant because they train small models from filtered model outputs, exactly our setting. The systems reviewed here are not enough for our task, for one measured reason. Their pass criteria are proxies. Execution success accepts queries that run and compute the wrong thing. Agreement accepts consistent errors. Gold-answer matching accepts right answers reached by invalid programs. During our own harvest, 86.7 percent of generated bridge plans passed every deterministic check while only 52.3 percent matched the oracle. A filter blind to that stratum feeds a third of its pool with structurally valid wrong programs and never knows. We retain that stratum with its verifier diagnoses for failure analysis and for controlled negative-training experiments, and we report where it leaks.

Agents over structured knowledge. Pangu (Gu et al., 2023) constrains an LLM to discriminate among logical-form extensions enumerated from the graph. ChatKBQA (Luo et al., 2024a) generates a form and grounds it by retrieval. RoG (Luo et al., 2024b) plans relation paths and reasons over them. StructGPT (Jiang et al., 2023) gives a frozen model iterative read access through fixed interfaces. These systems share our premise that the structure, not the model, is the ground. They are not enough for two reasons. Their training pipelines, as reported, draw no supervision from verifier rejections, so the training value of failure is discarded. And their reported evaluations do not score refusal. Questions that are ambiguous, unsupported by the schema, or matched by no records are first-class citizens of our benchmarks, with three trap families and a faithfulness-gated metric.

Executable table question answering. Classical semantic parsers on WikiTableQuestions reach 37 to 46 percent with weak supervision (Pasupat & Liang, 2015). TaPas (Herzig et al., 2020) and TaBERT (Yin et al., 2020) pre-train table encoders and reach the high forties to low fifties, with table-pretraining successors improving further (Liu et al., 2022; Jiang et al., 2022). Binder (Cheng et al., 2023) and later LLM-with-code systems (Ye et al., 2023; Ni et al., 2023; Wang et al., 2024) pass 65 percent by generating SQL or Python against the table. These works calibrate our transfer numbers, and our answer-only result of 44.4 percent sits inside the classical band while our gold-program result of 51.8 exceeds TAPAS-large. They are not enough for our purpose because the program language is chosen for coverage, not for verifiability. Open-ended Python generation does not by itself provide a type checker that a second implementation can replay, a closed grammar that decoding can enforce, or abstention semantics. Our algebra buys those properties and pays for them in coverage, and Section 5.3 measures the price and then decomposes it.

Preference optimisation. Direct preference optimisation (Rafailov et al., 2023) and its variants are the standard way to use negatives. Likelihood displacement work (Razin et al., 2025) shows the objective is most destructive when chosen and rejected responses are similar. Our verifier produces exactly that regime,

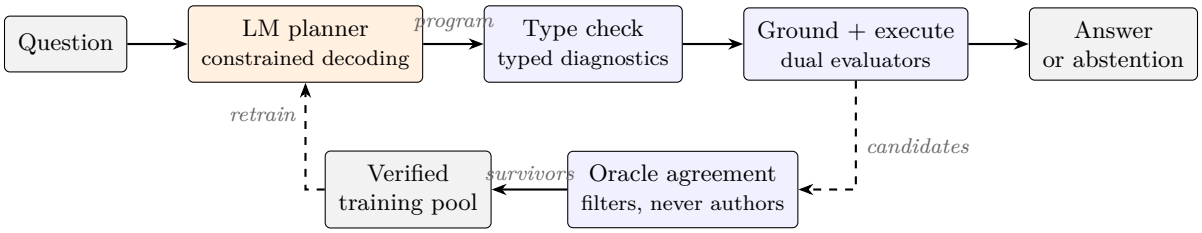

Figure 1: The system. A language model appears in one role, translating the question into a typed program under constrained decoding. Everything downstream is deterministic code. In the bootstrap loop, candidates that compile, ground, execute and match the independently computed oracle become training data; the oracle filters and never authors.

near-miss negatives one slot away from correct plans, and Section 6 reports the resulting hazard as a bounded negative finding rather than a method.

3 The Task and the System

Figure 1 shows the system. A language model appears in one role only. It translates a question into a program in a small typed algebra. Everything after that point is deterministic code, and only that code produces answers.

The algebra. Programs are trees over five value types, record sets, value sets, group tables, scalars, and booleans. Seventeen node types cover filtering with negation, disjunction and computed-set membership, projection, counting, guarded money summation, unique lookup, extremum records, grouping with count or sum metrics, group reduction by extremum or top-k, scalar arithmetic and comparison, set union, intersection and difference, and groupwise combination. A later transfer adds one generic node, the row-preserving arg-extremum, and nothing else. The grammar is closed but recursively compositional. Every node and enum is fixed, so a recursive type checker decides membership mechanically and its failures are typed diagnostics, while the nodes combine into well-typed trees never seen during design. Verification stays as mechanical as whitelist membership.

Deterministic execution. Two evaluators execute every tree. The runtime evaluator shares data loading with the production system. The independent evaluator rebuilds the record universe from raw files and shares no code. Both enforce the same stated conventions, deduplication on the contract identifier, exclusion of empty group keys, exact decimal money arithmetic under an additivity guard, and deterministic tie breaks. Agreement between them is how we measure, rather than assume, that oracles are right.

Decoding as format authority. At inference the algebra's grammar is compiled to a JSON schema and enforced by constrained decoding (Kwon et al., 2023). Every system we evaluate, including untrained bases, is format-locked, so accuracy differences measure planning and not JSON discipline. One diagnostic seals this point. Decoded freely, the untrained base emits question-grounded plans in a fluent self-invented schema on 98 of 100 questions but matches the executable contract on 3. The fine-tuned models match it on 98 and 99. Fine-tuning teaches the contract. Constrained decoding grants the contract to everyone, which is what makes the ladder a measure of planning quality.

Abstention. Refusal is an output, not a failure. A question may be ambiguous, may name a concept the schema does not carry, or may match no records. The system abstains through the same typed channel, and an empty result whose literals all trace to the question is treated as the data's answer rather than a defect. A repair loop exists but is gated. It consumes typed diagnostics from failed checks, proposes one revision, and re-enters the same checks. Abstentions and semantically meaningful empty results are final and are never repaired, a rule adopted after an ungated loop converted six correct refusals into confident wrong answers on a development slice.

The verified bootstrap. Training data is manufactured, not annotated. A generator authors a program, executes it for the answer under both evaluators, and only then renders a question surface. A harvest samples candidate programs from a teacher or from the student itself, executes them, and keeps a candidate only when every check passes and the answer matches the oracle. The oracle filters and never authors. No gold program or answer is copied into training text. What survives trains the next model. What fails is retained with its diagnosis for failure analysis and for controlled negative-training experiments, and the primary models are trained on verified survivors only. In the answer-only arms no human-authored gold program is used anywhere. The teacher proposes candidates but is never treated as an oracle, because deterministic execution and answer verification decide which trajectories survive.

4 Benchmarks and Evaluation Protocol

Procurement benchmark. The main benchmark contains 12,828 questions over a knowledge graph of 215,221 UK contract awards, built plan-first. A parameterised program is instantiated and executed, and only then is a question surface written, so every question carries an executable gold program. Splits separate plans, not surfaces. Train and test share zero underlying programs. Oracle correctness is measured, not assumed. A second evaluator, implemented from the raw files with no shared code, reproduces 99.88 percent of 14,770 audited answers, and the 18 residual disagreements are characterised individually. Three families of refusal questions, ambiguous, schema-unsupported and empty-result, are built in as traps with their cues preserved under paraphrase.

PACS. The procurement analytics challenge set evaluates the compositional planner on questions its training machinery never rendered. It contains 922 sealed test rows over 694 intent clusters, organised by seven task families, three depth levels, a seen and unseen exposure axis, and an answerability axis. Surfaces come from an independently written grammar. Isolation is enforced by five zero-overlap gates between training and test at the level of question text, logical signature, surface template, entity anchors, and program hash. The sealed test was evaluated once under the frozen primary configuration. A 93-row human audit preceded unsealing. Its one finding was a single systematic defect, a boolean-handling error in the empty-result class that had mislabelled 46 rows, corrected by offline re-execution before any number was read.

WikiTableQuestions. The transfer target is the standard benchmark of 22,033 crowd-written questions over 2,108 web tables (Pasupat & Liang, 2015). We use the official test split of 4,344 questions and the official evaluator. The test set was touched exactly once, under a frozen configuration whose commit hash and file checksums were recorded before launch, and it is treated as consumed thereafter. All development ran on folds of the training split that share no tables with any training pool.

Protocol. Every evaluation is a single model call at temperature zero under constrained decoding, with no repair unless a variant is named explicitly. Answerable questions score by type-aware match against the dual-verified oracle, or by the official evaluator on WikiTableQuestions. Refusal questions score under two declared metrics, an exact-status match and a faithfulness-gated variant in which an empty or multi-valued outcome counts only if every literal of the emitted program traces to the question. Paired systems are compared by exact McNemar tests on discordant pairs (McNemar, 1947). Teacher comparisons respect a measured provider noise floor of 1.1 points, and no single-run teacher delta below three points is claimed anywhere. Each student configuration is a single frozen training run. Paired significance therefore quantifies test-set disagreement between systems, not optimisation variance across seeds.

5 Experiments

5.1 Verified Bootstrap on the Home Benchmark

Figure 2(a) reports the five-system comparison on the held-out test set of 2,285 questions. The teacher is the strongest cloud configuration we could deploy. The hybrid systems use the cloud model for question understanding and a local 8B adapter for planning. The fully local systems use 8B LoRA adapters (Hu et al., 2022; Dettmers et al., 2023) for both stages, trained only on harvest survivors of the deterministic filter.

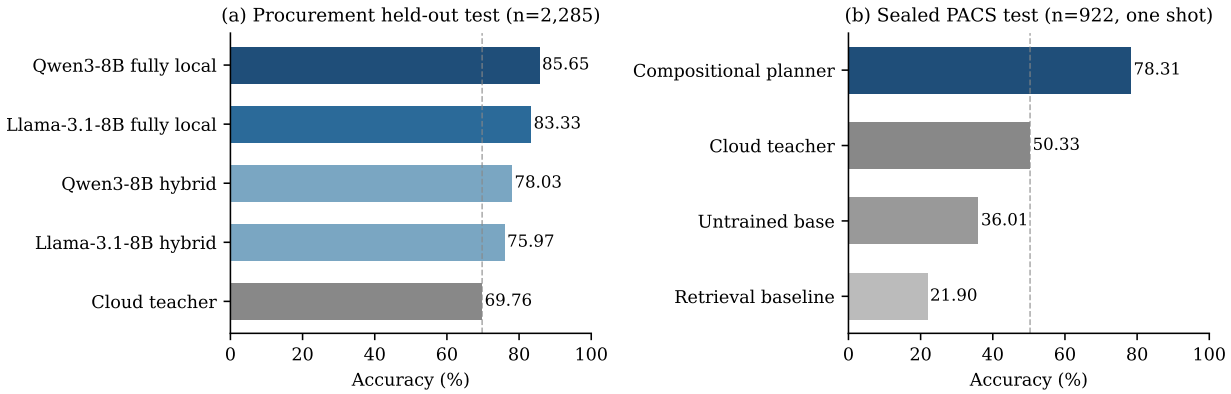

Figure 2: The two scoreboards. (a) On the held-out procurement test both hybrid and fully local students pass the cloud teacher that generated their training data, and the fully local systems pass the hybrids. (b) On the sealed PACS test, evaluated once, the ordering survives with a larger margin. Dashed lines mark the teacher.

Every pairing in the figure is significant at McNemar $p < 10^{-14}$. The fully local Qwen system (Qwen Team, 2025) exceeds its teacher by 15.89 points with a 95 percent confidence interval of 14.07 to 17.70. The result replicates on the second base model at 13.57 points. Because the teacher is served by a provider whose decoding is not seeded, we measured its variability with three identical development runs and obtained 72.6 plus or minus 1.1 points. The student beats each replicate separately, and no teacher comparison in this paper rests on a delta inside that floor.

Two controls locate where the gain comes from. Two retrieval baselines answer 31.5 and 28.8 percent of the development set, while an untrained 8B model forced through the executable scaffolding answers 61.5 percent, rising to 70.4 under the revised pipeline. Scaffolding, not retrieval, sets the floor, and training adds its gains on top of that floor. A format diagnostic separates the two remaining explanations. Decoded without constraints, the untrained base produces question-grounded plans in an invented schema on 98 of 100 questions and matches the executable contract on 3. The tuned models match it on 98 and 99. Fine-tuning contributes contract conformance and planning quality within it, and constrained decoding grants the contract to every system equally, so the table measures planning.

Two further slices report what the headline hides (Figure 3). First, moving from development to the 2,285-row test costs the hybrid systems 5.5 and 7.1 points but the fully local systems only 0.5 and 0.9, on both base models. Training on verified survivors appears to remove exactly the brittleness that shows up as dev-to-test decay. Second, on a composite hard slice of the development set the fully local student is flat, moving +0.8 points where the teacher loses 6 to 12 and a preference-tuned hybrid loses 7.7, so the student advantage is largest, 18 to 20 points, precisely where questions are hardest. Third, the bootstrap converges. A second self-harvest round lifts harvest yield from 76.6 to 86.3 percent, yet the evaluation gain is +0.4 points and not significant, so the verifier-gated loop has a natural ceiling on this task and we report it rather than iterate past it.

5.2 A Sealed Benchmark the System Did Not Author

The compositional planner was frozen and evaluated once on the 922 sealed PACS test rows. It scores 78.31 percent strict overall and 80.00 on the answerable subset, with a cluster bootstrap interval of 77.06 to 82.92. On the same rows the untrained base scores 36.01, a retrieval baseline 21.90, and the cloud teacher 50.33. The verified-bootstrap student exceeds its teacher by 27.98 points here. The comparison is a system-level one between deployable configurations, not a model comparison under equal decoding, because the teacher's API offers no constrained decoding and 23.1 percent of its rows fail on format alone. Figure 4 collects the audits that attack the headline from both directions.

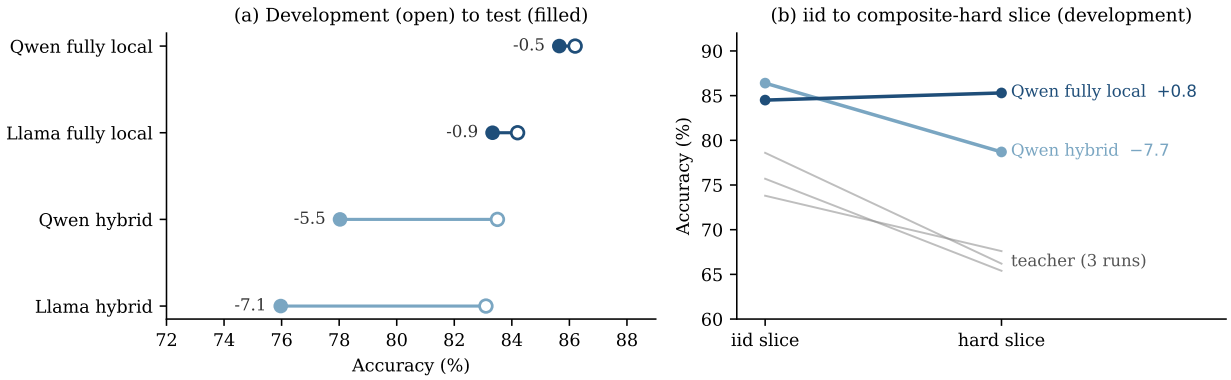

Figure 3: What the headline hides. (a) Development-to-test movement per system. Hybrid systems decay steeply on both bases while fully local systems barely move, so verified-survivor training removes the brittleness that surfaces as dev-to-test decay. (b) On the composite hard slice the teacher and the preference-tuned hybrid fall while the fully local student is flat, so its advantage is largest where questions are hardest.

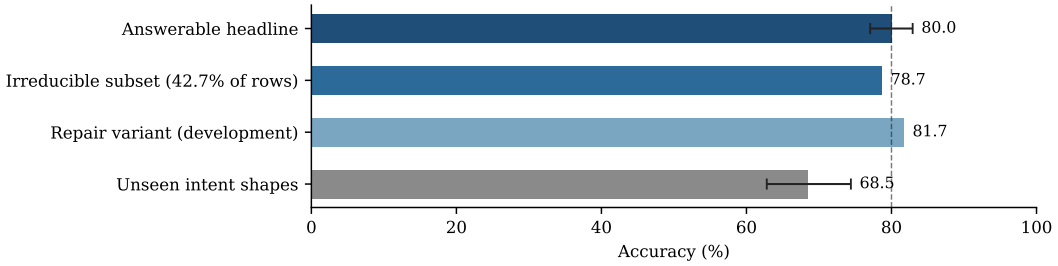

Figure 4: PACS integrity audits around the answerable headline (dashed line, cluster bootstrap interval shown). The planner loses only 1.3 points on the shortcut-resistant irreducible subset, gains 1.7 from a typed-feedback repair variant that is reported but not primary, and pays 11.5 points (interval 5.6 to 17.2) on intent shapes never demonstrated in training.

Two audits attack this headline from opposite directions. Deleting each oracle-program predicate in turn and re-executing shows that 42.7 percent of answerable test rows are irreducible, meaning every predicate is load bearing. On that shortcut-resistant subset the planner scores 78.7, within 1.3 points of its answerable headline, so the number is not inflated by questions a partial program could answer by accident. In the other direction, a typed-feedback repair loop that only ever acts on hard failures adds 1.7 points on development rows while leaving every refusal untouched, and we report the single-call number as primary.

The unseen-exposure axis bounds generalisation honestly. Intent shapes absent from training cost 11.5 points, with an interval of 5.6 to 17.2. Composition over demonstrated constructs transfers. Constructions never demonstrated remain measurably harder, and the two weakest families, scope binding and universal quantification, are named rather than averaged away.

5.3 Transfer to WikiTableQuestions

Moving the system to web tables required a new data loader with typed views, a column-aware value linker, one generic operator, and nothing else. All seventeen original operators, the type checker, and both evaluators transferred unchanged. The linker was admitted by a four-arm ablation in which real cell candidates gain 4.0 points over the schema baseline while random cells lose 1.7, so the gain is grounding rather than added context.

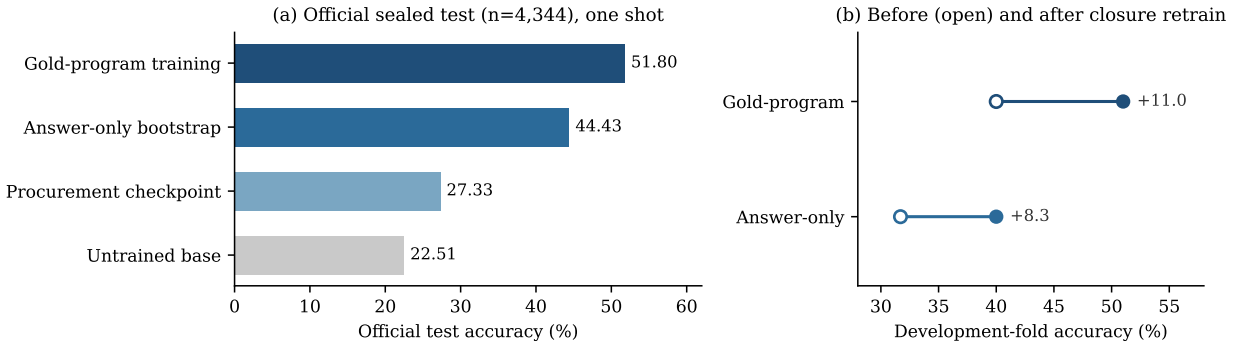

Figure 5: WikiTableQuestions transfer. (a) The four rungs on the official sealed test, touched once; each step is significant at $p < 5 \times 10^{-18}$. (b) Retraining after representation-and-compilation closure lifts the development fold in both supervision regimes, so capability tracks the coverage boundary.

Table 1: Where the WikiTableQuestions gap lives. Stage-by-stage decomposition against 11,276 gold SQL programs. Each rate conditions on the stage above it.

Stage	Rate
Gold programs expressible in the algebra (before closure)	53–55%
after translator and loader closure, zero regressions	61.0%
Translated programs matching the reference denotation	92–93%
Executed class-A programs recovering the target answer	94.1%
Gold SQL annotations reproducing the benchmark answer	88.9%

On the official test set, evaluated once under a recorded manifest, the four rungs of Figure 5(a) are each significant at $p < 5 \times 10^{-18}$. The checkpoint carries a 4.8-point zero-shot prior, and the recipe carries 17 to 24.5 points beyond it. The answer-only figure matches classical weakly-supervised parsers on this benchmark, the gold-program figure exceeds early table-pretrained baselines such as TaPas (Herzig et al., 2020), and neither approaches recent specialised systems (Cheng et al., 2023; Ye et al., 2023; Wang et al., 2024), a gap the next paragraph explains rather than excuses.

A differential audit against 11,276 gold SQL programs (Shi et al., 2020) separates where the remaining gap actually lives. Table 1 decomposes the pipeline stage by stage, and Figure 6 shows the resulting attribution over the gold-program universe. The algebra expresses 53 to 55 percent of gold programs. Translated programs match the reference denotation 92 to 93 percent of the time, and executed class-A programs recover the target answer in 94.1 percent of cases, so performance is coverage limited, not execution limited. By comparison, the gold SQL annotations themselves reproduce the benchmark answer in only 88.9 percent, so the annotation layer is noisy and is not treated as an absolute oracle. A closure pass that lowers SQL constructs onto existing nodes and adds typed loader views raises coverage to 61.0 percent with zero regressions. One seventh of the apparent expressiveness gap was compiler and representation debt. Retraining after closure lifts the development fold in both supervision regimes (Figure 5b), so capability tracks the coverage boundary.

6 Discussion and Limitations

The primary claim is about a task and the system that solves it. The teacher comparison inside that claim is scoped carefully. The teacher operates zero-shot while the students are tuned in domain, so the scoreboard shows that a verified bootstrap beats deploying the teacher it was distilled from, not that an 8B model outranks its teacher in general ability. PACS, where both sides face questions neither authored, shows the gap survives on independent surfaces.

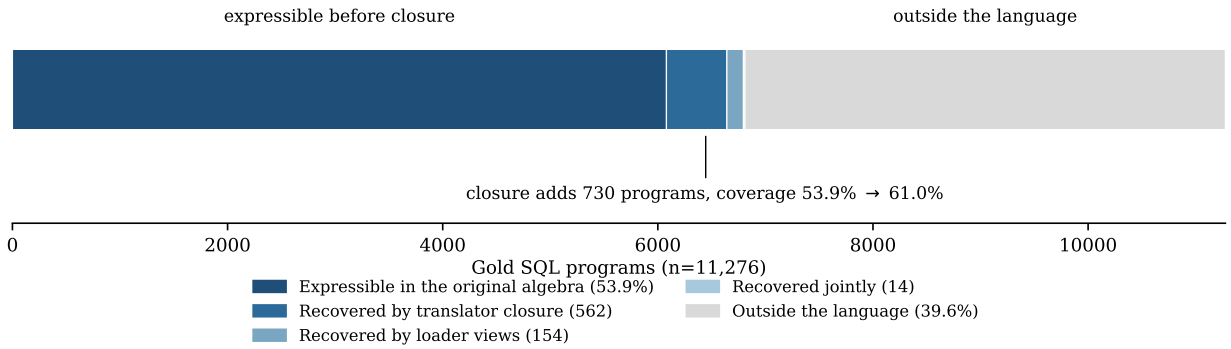

Figure 6: Attribution over the 11,276 gold SQL programs. The closure pass recovers 562 questions through the translator alone, 154 through typed loader views alone and 14 jointly, moving coverage from 53.9 to 61.0 percent; 4,466 remain outside the language, and the two largest remaining families are specified as typed extensions and deliberately not built mid-evaluation.

Refusal did not transfer free. On a paired legacy comparison the compositional planner wins the answerable subset by 4.4 points yet loses explicit abstention badly, because its training mix carried 37 refusal demonstrations. Under our pre-registered bar of no key-bucket regression, we make no retirement claim for the older system. Competence follows demonstrations, and refusal is a competence.

Preference optimisation was hazardous in exactly the regime our filter creates. Verifier rejects are near-miss negatives, one slot away from correct plans, and suppressing them dragged down adjacent correct modes on one base while cleaner pools made it worse (Rafailov et al., 2023; Razin et al., 2025). We report the mechanism and use additive distillation instead.

The transfer numbers are coverage limited. The algebra expresses roughly half of gold programs, the planner reaches 85.3 percent of what the gold subset can express, and the closure pass shows a seventh of the missing half was compiler debt. The two largest remaining gaps, general expression predicates and ordered-relation navigation, are specified as typed extensions and deliberately not built, because widening a language mid-evaluation would dissolve the boundary that makes the measurements interpretable.

Four further limits bound the claims. The home benchmark is self-built, with its integrity measured rather than assumed, and its question surfaces are model-generated under style controls, which is not user language. The WikiTableQuestions test set is consumed, so future work on that benchmark reports development folds or a new holdout. Probe evidence of composition beyond demonstrated constructs is bounded, 11.5 points of unseen-exposure cost. The evidence remains confined to executable structured-data analytics over one procurement graph and one table benchmark.

7 Conclusion

UK public procurement is public data that the public cannot reliably analyse in natural language. We defined that task end to end, a convention-explicit knowledge graph, a program-first benchmark with refusal as scored behaviour, a typed compositional program language, and oracles whose correctness is measured by an independent second implementation. On that foundation a verified bootstrap, deterministic checks deciding what a model may learn from, trains an 8B local system to 85.65 percent, past the cloud teacher that generated its training data, and to 78.31 on a sealed challenge set neither system authored. Rebuilt on web tables at the cost of a data layer, the same recipe turns a 22.5 percent floor into 44.4 without a single gold program and 51.8 with them. Language coverage is now the dominant measured bottleneck, while program discovery remains incomplete, and the audits in this paper are what make that distinction measurable.

References

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. In *International Conference on Learning Representations (ICLR)*, 2023.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yu Gu, Xiang Deng, and Yu Su. Don't generate, discriminate: A proposal for grounding language models to real-world environments. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Zhengbao Jiang, Yi Mao, Pengcheng He, Graham Neubig, and Weizhu Chen. OmniTab: Pretraining with natural and synthetic data for few-shot table-based question answering. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *International Conference on Learning Representations (ICLR)*, 2022.

Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, and Anh Tuan Luu. ChatKBQA: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. In *Findings of the Association for Computational Linguistics: ACL*, 2024a.

Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*, 2024b.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning (ICML)*, 2023.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Noam Razin, Sadhika Malladi, Adithya Bhaskar, Danqi Chen, Sanjeev Arora, and Boris Hanin. Unintentional unalignment: Likelihood displacement in direct preference optimization. In *International Conference on Learning Representations (ICLR)*, 2025.

Tianze Shi, Chen Zhao, Jordan Boyd-Graber, Hal Daumé III, and Lillian Lee. On the potential of lexico-logical alignments for semantic parsing to SQL queries. In *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

Ahmet Soylu, Oscar Corcho, Brian Elvesæter, Carlos Badenes-Olmedo, Tom Blount, Francisco Yedro Martínez, Matej Kovacic, Matej Posinkovic, Ian Makgill, Chris Taggart, Elena Simperl, Till C. Lech, and Dumitru Roman. TheyBuyForYou platform and knowledge graph: Expanding horizons in public procurement with open linked data. *Semantic Web*, 13(2):265–291, 2022.

Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations (ICLR)*, 2024.

Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Jipeng Zhang, Haolin Yang, Kehao Miao, Ruiyuan Zhang, Renjie Pi, Jiahui Gao, and Xiaofang Zhou. ExeSQL: Self-taught text-to-SQL models with execution-driven bootstrapping for SQL dialects. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024.

Yushan Zhu, Wen Zhang, Long Jin, Mengshu Sun, Ling Zhong, Zhiqiang Liu, Juan Li, Lei Liang, Chong Long, Chao Deng, and Junlan Feng. Self-correction distillation for structured data question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

A Operator Algebra Reference

Programs are trees over seven value types. RECORDS is a subset of the flat record universe, VALUES a distinct set of scalars, GROUPS a mapping from group key to number, RANKING an ordered list of key–number pairs, and NUMBER, VALUE and BOOL are scalars. Table 2 lists the closed node inventory with type signatures. Depth is capped at 16 and node count at 64. Every check failure raises a typed diagnostic with a path into the tree, which is what the gated repair loop consumes.

Table 2: Node inventory. The seventeen procurement nodes plus the one node added for the WikiTableQuestions transfer (marked †). Predicates inside `filter` support base constraints (`eq`, `in`, `contains`, `exists`, `gte`, `lte`), negation, disjunction, and computed-set membership (`in_expr`, a semijoin or antijoin on a subtree).

Node	Signature	Meaning
`filter`	preds → RECORDS	subset by predicate conjunction
`values`	RECORDS → VALUES	distinct field values
`count`	RECORDS → NUMBER	record count
`size`	VALUES → NUMBER	set cardinality
`sum`	RECORDS → NUMBER	guarded money summation
`exists`	RECORDS → BOOL	non-emptiness
`select`	RECORDS → VALUE	unique field lookup
`extreme`	RECORDS → VALUE	argmax/argmin record
`groupby`	RECORDS → GROUPS	grouped count or sum
`argext`	GROUPS → VALUE	extremum group key
`top`	GROUPS → RANKING	top-k groups
`num`	literal → NUMBER	numeric literal
`combine`	NUMBER^2 → NUMBER/BOOL	arithmetic or comparison
`vcompare`	VALUE → BOOL	scalar comparison, date-aware
`setop`	VALUES^2 → VALUES	union, intersection, difference
`gcombine`	GROUPS^2 → GROUPS	groupwise combination
`keys_where`	GROUPS → VALUES	keys passing a threshold
`extreme_rows`†	RECORDS → RECORDS	row-preserving arg-extremum

B Audit Protocols and Supplementary Results

Dual-oracle audit. The independent evaluator was implemented from the raw releases with no shared code and replays every benchmark answer. Agreement is 99.88 percent over 14,770 audited answers. All 18 residual disagreements were opened by hand and characterised; none affects a reported comparison.

PACS isolation gates. Five zero-overlap gates hold between training and sealed test at the level of question text, logical signature, surface template, entity anchors, and program hash. A 93-row human audit preceded unsealing and found one systematic defect, a boolean-handling error in the empty-result class affecting 46 rows, corrected by offline re-execution before any number was read.

Constraint-necessity audit. Each predicate of each oracle program is deleted in turn and the program re-executed. A row is irreducible when every deletion changes the answer. 42.7 percent of answerable sealed rows are irreducible, and the planner scores 78.7 on that subset against 80.00 on the full answerable set.

Trap-cue disclosure. Nineteen of 2,285 test rows string-match cue phrases derived from development refusal traps. Excluding them moves every headline by at most 0.21 points and changes no ranking and no significance verdict.

Method ladder. Table 3 reports the development-set ladder over training methods and bases under the revised pipeline, and the harvest yields that feed them. Preference optimisation helps one base and hurts the other, consistent with the near-miss hazard discussed in Section 6; the deployed fully local systems use the

best arm per base. Self-harvest yield exceeds teacher harvest yield on both bases, and a second self-harvest round raises yield further, 76.6 to 86.3 percent on answerable questions, while adding only +0.4 evaluation points, not significant, which is where we stop the loop.

Table 3: Development ladder (n=260) and harvest yields. Yields are data-engine acceptance rates on answerable questions, a different metric from system accuracy, and are never compared against it.

Arm	Qwen3-8B	Llama-3.1-8B
Zero-shot scaffolded	70.4	60.0
SFT	81.2	83.1
Rejection-sampling FT	81.5	82.7
DPO	83.5	77.3
Fully local (best arm)	86.2	84.2
Teacher harvest yield	65.5	
Self-harvest yield, round 1	76.6	78.1
Self-harvest yield, round 2	86.3	–

C Configuration

Students are Qwen3-8B (Qwen Team, 2025) and Llama-3.1-8B, adapted with rank-64 LoRA (Hu et al., 2022) under 4-bit QLoRA quantisation (Dettmers et al., 2023) in LLaMA-Factory (Zheng et al., 2024), and served with vLLM (Kwon et al., 2023) under guided JSON decoding compiled from the algebra grammar. All evaluations run at temperature zero with a single call per question. The WikiTableQuestions official run was executed once under a recorded manifest of commit hash, file checksums, decoding parameters and adapter identity, and the test set is treated as consumed. Full training configurations, prompts and manifests ship with the code.